
Gestión de Alertas — SIGAI-SES

Securitas Colombia S.A.



Gestión de Alertas — SIGAI-SES

Version 1.0.0 Status En Produccion FastAPI 0.136+ React 18 PostgreSQL / MySQL
Python 3.12+

Guía completa del **módulo de alertas** de SIGAI-SES: arquitectura, modelo de datos, motor de reglas, endpoints REST, panel de React y programación automática.

1. Arquitectura del módulo de alertas

1.1 Estructura actual (FastAPI + React + SQLAlchemy)

El módulo está implementado de forma transversal en el stack:

Backend/

└─ app/

├─ models/alerts.py # Modelos Alert y AlertRule (SQLAlchemy)
├─ schemas/alerts.py # Schemas Pydantic (AlertRead, AlertUpdate, AlertEstad)
├─ api/endpoints/alerts.py # Endpoints REST /api/v1/alerts
├─ crud/crud_alerts.py # Lógica de BD + motor de reglas (evaluar_alertas)
└─ core/scheduler.py # APScheduler – evaluación automática (AsyncIOScheduT

Frontend/src/

├─ services/alerts.ts # API service (summary del dashboard)
├─ hooks/useAlerts.ts # React Query hooks (CRUD + estado de alertas)
├─ hooks/useAlertsSummary.ts # React Query hook del resumen del navbar
└─ pages/Alerts.tsx # Centro de alertas (tabla + tarjetas)

2. Modelo de datos

2.1 Tabla alerts

Ver DDL de la tabla alerts

```
CREATE TABLE alerts (  
  id SERIAL PRIMARY KEY,  
  created_at TIMESTAMP DEFAULT NOW(),  
  updated_at TIMESTAMP,  
  resolved_at TIMESTAMP,  
  tipo VARCHAR(50) NOT NULL,          -- stock_bajo,  
  ↪ garantía_vencida, manual  
  prioridad alerta_prioridad NOT NULL DEFAULT 'media', --  
  ↪ critica|alta|media|baja  
  estado alerta_estado NOT NULL DEFAULT 'activa', --  
  ↪ activa|reconocida|resuelta|ignorada  
  titulo VARCHAR(200) NOT NULL,  
  descripcion TEXT,  
  item_id INTEGER NOT NULL REFERENCES items(id_item),  
  item_nombre VARCHAR(200),  
  valor_actual DECIMAL(10,2),  
  valor_umbral DECIMAL(10,2),  
  unidad VARCHAR(20),  
  asignado_a INTEGER REFERENCES usuarios(id_usuario),  
  solucion TEXT  
);
```

2.2 Tabla alert_rules (reglas configurables)

Ver DDL de la tabla alert_rules

```
CREATE TABLE alert_rules (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  tipo VARCHAR(50) NOT NULL,  
  activa BOOLEAN NOT NULL DEFAULT TRUE,  
  prioridad regla_prioridad NOT NULL,  
  condicion TEXT NOT NULL,          -- condición almacenada como JSON  
  descripcion TEXT,  
  cooldown_h INTEGER DEFAULT 24  
);
```

3. Motor de reglas

Archivo: Backend/app/crud/crud_alerts.py — función evaluar_alertas(db)

[!NOTE] El motor recorre los items y garantías y genera alertas automáticamente. Solo crea una alerta si **no existe ya una activa o reconocida** para el mismo item/tipo (evita duplicados).

3.1 Reglas implementadas

Tipo	Disparador	Prioridad
stock_bajo	cantidad_actual <= stock_minimo (StockBulk vs Item)	crítica
garantia_vencida	Caso ENVIADO_PROVEEDOR con fecha_envio con más de 15 días	alta

3.2 Comportamiento

- Recorre items JOIN stock_bulk donde cantidad_actual <= stock_minimo y crea alerta crítica si no existe una activa/reconocida.
- Recorre garantías en estado ENVIADO_PROVEEDOR con fecha_envio anterior a hace 15 días y crea alerta de alta prioridad.
- Registra todo en una transacción (db.commit() / db.rollback() en caso de error) y usa el logger estructurado.

4. Endpoints REST (FastAPI)

[!TIP] Todos los endpoints viven bajo el prefijo `/api/v1/alerts` y retornan respuestas en formato JSON estándar.

Ver endpoints disponibles

Metodo	Ruta	Descripción
GET	<code>/api/v1/alerts/</code>	Lista alertas (paginado, filtrable por estado/prioridad/tipo)
GET	<code>/api/v1/alerts/alerts</code>	Vista de listado de alertas
GET	<code>/api/v1/alerts/summary</code>	Resumen para el Dashboard y navbar
PATCH	<code>/api/v1/alerts/{id}</code>	Cambia estado: reconocida, resuelta, ignorada (con notas, valor_actual, solución, asignación)
DELETE	<code>/api/v1/alerts/{id}</code>	Elimina alerta
POST	<code>/api/v1/alerts/event</code>	Dispara manual del motor de reglas
POST	<code>/api/v1/alerts/</code>	Crear alerta manual

Ejemplo de respuesta GET `/alerts/summary`:

```
{
  "total": 8,
  "stock": [ { "id": 12, "title": "Stock crítico: Cámara Dome", "count": 3.0
    ↪   } ],
  "garantias": [ { "id": 5, "title": "Garantía estancada: GSES-045" } ]
}
```

5. Panel de alertas en React (frontend)

5.1 Página `Alerts.tsx`

El centro de alertas se encuentra en `Frontend/src/pages/Alerts.tsx`. Incluye:

- **Vista mixta:** tabla paginada + tarjetas de resumen.

- **Filtros:** por estado (activa/reconocida/resuelta/ignorada) y paginación.
- **Acciones:** actualizar estado con notas, valor actual y solución; crear y eliminar alertas.
- **React Query:** hooks `useAlerts`, `useCreateAlert`, `useUpdateAlertEstado`, `useDeleteAlert` (staleTime 5 min, invalidación automática de la caché ["alerts"]).

5.2 Badge en el Navbar

El conteo de alertas activas se muestra en el Navbar (dentro de `Fusion.tsx`) como un badge sobre el icono de campana. Usa el hook `useDashboardAlerts()` que llama a `GET /alerts/summary` (caché de 5 min).

5.3 Dashboard

Las alertas se muestran en el Dashboard principal como tarjetas de resumen que enlazan al centro de alertas, junto con el widget “SIGAI-SES AI” que predice **stock por agotarse** y **garantías por vencer** (endpoint `/analytics/predictions`).

6. Scheduler automático de evaluación

[!IMPORTANT] El scheduler ejecuta `evaluar_alertas()` cada **30 minutos**, y también una vez inmediatamente al arrancar. Se integra con el ciclo de vida de FastAPI mediante el patrón `lifespan` usando **`APScheduler AsyncIOScheduler`** (no `BackgroundScheduler`).

Ver configuración real del scheduler

Archivo `Backend/app/core/scheduler.py`:

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler
from apscheduler.triggers.interval import IntervalTrigger

scheduler = AsyncIOScheduler()

async def evaluate_alerts_job():
    from app.crud.crud_alerts import evaluar_alertas
    # ... crea engine y sesión asíncrona, ejecuta evaluar_alertas(db) ...
```

```
def start_scheduler():
    if not scheduler.running:
        scheduler.add_job(
            evaluate_alerts_job,
            trigger=IntervalTrigger(minutes=30),
            id="evaluate_alerts",
        )
    scheduler.start()
    scheduler.add_job(evaluate_alerts_job, trigger="date",
                      id="evaluate_alerts_startup") # evalúa al inicio
```

Integración con lifespan en app/main.py:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    start_scheduler()
    yield
    stop_scheduler()
```

7. Estados de alerta

Estado	Significado	Acción
activa	Recién generada, sin gestionar	Reconocer / resolver / ignorar
reconocida	En revisión por un responsable	Resolver / ignorar
resuelta	Atendida (se marca resolved_at)	Histórico
ignorada	Descartada (se marca resolved_at)	Histórico

8. KPIs del módulo de alertas

[!TIP] El dashboard permite medir la *salud* del proceso de alertas.

KPI	Descripción
Total activas	Alertas sin gestionar (resumen del navbar/dashboard)
Stock crítico	Items bajo el mínimo, con <code>valor_actual</code> vs <code>valor_umbra</code>
Garantías estancadas	Casos sin avance por más de 15 días
Tiempo de resolución	<code>resolved_at</code> - <code>created_at</code> por alerta

9. Checklist de implementación

Base (implementado)

- ☑ Tabla `alerts` y `alert_rules` en la BD (Alembic)
- ☑ Modelos SQLAlchemy `Alert` y `AlertRule`
- ☑ Motor de reglas `evaluar_alertas()` (stock bajo + garantías estancadas)
- ☑ Scheduler (APScheduler AsyncIOScheduler) cada 30 minutos + evaluación al arrancar
- ☑ Deduplicación: no re-crea alertas activas/reconocidas del mismo item/tipo

API REST (implementado)

- ☑ Endpoints: GET listado, GET summary, PATCH estado, DELETE, POST evaluar, POST crear

Frontend (implementado)

- ☑ Centro de alertas `Alerts.tsx` con tabla paginada y filtros
- ☑ Badge de alertas en el Navbar (campana)
- ☑ Resumen de alertas en el Dashboard

Mejoras futuras (pendiente)

- ☐ Configuración visual de reglas (toggle + umbrales) en el frontend
 - ☐ Historial detallado y exportación Excel de alertas
 - ☐ Escalamiento automático por SLA (críticas sin reconocer > 2h)
 - ☐ Notificaciones push / WhatsApp / correo para alertas críticas
-

Documento actualizado: Agosto 2026 — v1.1 — Compatible con FastAPI + React + SQLAlchemy + APScheduler